



EXVUL

SMART CONTRACT **AUDIT REPORT**

PayMentNFT Smart Contract

MARCH 2026

Contents

1. EXECUTIVE SUMMARY	4
1.1 Methodology	4
2. FINDINGS OVERVIEW	7
2.1 Project Info And Contract Address	7
2.2 Summary	7
2.3 Key Findings	8
3. DETAILED DESCRIPTION OF FINDINGS	9
3.1 Duplicate Payments Leading to Zero Fund Risk	9
3.2 Audit of Reward Calculation Logic Defect Leading to User Financial Loss	11
3.3 Financial Loss Due to Unexpected TotalSurplus Status Update	13
3.4 drawDynamic() Fails to Update totalSurplus[] Status	15
3.5 Missing userReward Update Leading to Transaction Revert	17
3.6 addUserList() lacks deduplication check	20
3.7 Unchecked ERC20 Transfer Return Values	22
4. CONCLUSION	23
5. APPENDIX	24
5.1 Basic Coding Assessment	24
5.1.1 Apply Verification Control	24
5.1.2 Authorization Access Control	24
5.1.3 Forged Transfer Vulnerability	24
5.1.4 Transaction Rollback Attack	25
5.1.5 Transaction Block Stuffing Attack	25
5.1.6 Soft Fail Attack Assessment	25
5.1.7 Hard Fail Attack Assessment	26
5.1.8 Abnormal Memo Assessment	26
5.1.9 Abnormal Resource Consumption	26
5.1.10 Random Number Security	27
5.2 Advanced Code Scrutiny	27
5.2.1 Cryptography Security	27
5.2.2 Account Permission Control	27
5.2.3 Malicious Code Behavior	28
5.2.4 Sensitive Information Disclosure	28
5.2.5 System API	28

6. DISCLAIMER	29
7. REFERENCES	30
8. About Exvul Security	31

1. EXECUTIVE SUMMARY

ExVul Web3 Security was engaged by **PayMentNFT** to review smart contract implementation. The assessment was conducted in accordance with our systematic approach to evaluate potential security issues based upon customer requirement. The report provides detailed recommendations to resolve the issue and provide additional suggestions or recommendations for improvement.

The outcome of the assessment outlined in chapter 3 provides the system's owners a full description of the vulnerabilities identified, the associated risk rating for each vulnerability, and detailed recommendations that will resolve the underlying technical issue.

1.1 Methodology

To standardize the evaluation, we define the following terminology based on OWASP Risk Rating Methodology [10] which is the gold standard in risk assessment using the following risk models:

- **Likelihood:** represents how likely a particular vulnerability is to be uncovered and exploited in the wild.
- **Impact:** measures the technical loss and business damage of a successful attack.
- **Severity:** determine the overall criticality of the risk.

Likelihood can be: High, Medium and Low and impact are categorized into: High, Medium, Low, Informational. Severity is determined by likelihood and impact and can be classified into five categories accordingly: Critical, High, Medium, Low, Informational shown in table 1.1.

		Informational	Low	Medium	High
Likelihood	High	INFO	MEDIUM	HIGH	CRITICAL
	Medium	INFO	LOW	MEDIUM	HIGH
	Low	INFO	LOW	LOW	MEDIUM
		IMPACT			

Table 1.1 Overall Risk Severity

To evaluate the risk, we will be going through a list of items, and each would be labelled with a severity category. The audit was performed with a systematic approach guided by a comprehensive assessment list carefully designed to identify known and impactful security issues. If our tool or analysis does not identify any issue, the contract can be considered safe regarding the assessed item. For any discovered issue, we might further deploy contracts on our private test environment and run tests to confirm the findings. If necessary, we would additionally build a PoC to demonstrate the possibility of exploitation. The concrete list of check items is shown in Table 1.2.

- **Basic Coding Bugs:** We first statically analyze given smart contracts with our proprietary static code analyzer for known coding bugs, and then manually verify (reject or confirm) all the issues found by our tool.
- **Code and business security testing:** We further review business logics, examine system operations, and place DeFi-related aspects under scrutiny to uncover possible pitfalls and/or bugs.
- **Additional Recommendations:** We also provide additional suggestions regarding the coding and development of smart contracts from the perspective of proven programming practices.

Category	Assessment Item
Basic Coding Assessment	• Apply Verification Control • Authorization Access Control • Forged Transfer Vulnerability • Forged Transfer Notification • Numeric Overflow • Transaction Rollback Attack • Transaction Block Stuffing Attack • Soft Fail Attack • Hard Fail Attack • Abnormal Memo • Abnormal Resource Consumption • Secure Random Number

Advanced Source Code Scrutiny	• Asset Security • Cryptography Security • Business Logic Review • Source Code Functional Verification • Account Authorization Control • Sensitive Information Disclosure • Circuit Breaker • Blacklist Control • System API Call Analysis • Contract Deployment Consistency Check • Abnormal Resource Consumption
Additional Recommendations	• Semantic Consistency Checks • Following Other Best Practices

Table 1.2: The Full List of Assessment Items

To better describe each issue we identified, we categorize the findings with Common Weakness Enumeration (CWE-699) [14], which is a community-developed list of software weakness types to better delineate and organize weaknesses around concepts frequently encountered in software development.

2. FINDINGS OVERVIEW

2.1 Project Info And Contract Address

Project Name	Audit Time	Language
PayMentNFT	16/03/2026 - 18/03/2026	Solidity

Repository

PayMentNFT.sol

Commit Hash

72eb03fd39129538ae2f2a9d1945ef84

2.2 Summary

Severity	Found
CRITICAL	1
HIGH	3
MEDIUM	1
LOW	2
INFO	0



2.3 Key Findings

Severity	Findings Title	Status
CRITICAL	Duplicate Payments Leading to Zero Fund Risk	Fixed
HIGH	Audit of Reward Calculation Logic Defect Leading to User Financial Loss	Fixed
HIGH	Financial Loss Due to Unexpected TotalSurplus Status Update	Fixed
HIGH	drawDynamic() Fails to Update totalSurplus[] Status	Fixed
MEDIUM	Missing userReward Update Leading to Transaction Revert	Fixed
LOW	addUserList() lacks deduplication check	Fixed
LOW	Unchecked ERC20 Transfer Return Values	Fixed

Table 2.3: Key Audit Findings

3. DETAILED DESCRIPTION OF FINDINGS

3.1 Duplicate Payments Leading to Zero Fund Risk

SEVERITY:**CRITICAL****STATUS:****Fixed****PATH:**

PayMentNFT.sol

DESCRIPTION:

The `recaption()` function calculates the reward due to the user, splits it into multiple portions, and transfers the result. However, after paying `toUser`, the function transfers `userReward` to the same recipient again, which duplicates the payout.

```
function recaption(uint256 _tokenId,uint256 _round) external
nonReentrant{
    // ...
    if(_round==round){
        // ...
        if(userReward>0){
            toUp(msg.sender,userReward);
            toUserList(userReward);
            totalState[msg.sender] = totalState[msg.sender] +
userReward;
            uint256 toBuyAbb = userReward / 10;
            if (address(buyAbb) != address(0)){
                gotoBuyAbb(toBuyAbb, address(this));
            }
            uint256 getArt = userReward / 20;
            uint256 toUser = userReward - toBuyAbb - getArt;
            usdt.transfer(toArt, getArt);
            usdt.transfer(msg.sender, toUser);
            usdt.transfer(msg.sender, userReward);
            emit Draw(msg.sender, userReward,
round,block.timestamp,1);
        }
        lastUpdateTime[_tokenId] = block.timestamp;
    }
}
```

```
} // ...
```

IMPACT:

Users will receive double the reward, potentially causing contract funds to eventually become zero.

RECOMMENDATIONS:

Remove the second user transfer so the function only sends the net amount once.

```
- usdt.transfer(msg.sender, userReward);
```

3.2 Audit of Reward Calculation Logic Defect Leading to User Financial Loss

SEVERITY:**HIGH****STATUS:****Fixed****PATH:**

PayMentNFT.sol

DESCRIPTION:

The `toUp()` function calculates the actual reward for inviters. When `rewardForUp > userSurplus[up][round]`, the function overwrites `rewardForUp` itself. Because the same variable is reused across the loop, one capped payout reduces the reward available for later inviters as well.

```
function toUp(address _add,uint256 _reward) internal{
    uint256 rewardForUp = _reward * 5 / 100;
    address downAdd = _add;
    for(uint256 i = 1;i<=9;i++){
        address up = superiorAddress[downAdd];
        if (up == address(0)) break;
        if(buySize[up][round] >= i){
            if (rewardForUp>userSurplus[up][round]){
                rewardForUp = userSurplus[up][round];
            }
            if (rewardForUp > 0) {
                userDynamic[up][round] = userDynamic[up][round] +
rewardForUp;
            }
        }
        downAdd = up;
    }
}
```

IMPACT:

Overriding the `rewardForUp` variable may prevent users who are supposed to receive the rewards from getting the rewards they are entitled to.

RECOMMENDATIONS:

Use a separate local variable to track the capped reward of the current inviter while keeping the original `rewardForUp` unchanged for subsequent iterations.

```
function toUp(address _add,uint256 _reward) internal{
    uint256 rewardForUp = _reward * 5 / 100;
    address downAdd = _add;
    for(uint256 i = 1;i<=9;i++){
        address up = superiorAddress[downAdd];
        if (up == address(0)) break;
        if(buySize[up][round] >= i){
+           uint256 currentReward;
-           if (rewardForUp>userSurplus[up][round]){
+               rewardForUp = userSurplus[up][round];
-               currentReward = userSurplus[up][round];
+           }
+           } else {
+               currentReward = rewardForUp;
+           }
-           if (rewardForUp > 0) {
-               userDynamic[up][round] = userDynamic[up][round] +
rewardForUp;
+           if (currentReward > 0) {
+               userDynamic[up][round] = userDynamic[up][round] +
currentReward;
            }
        }
        downAdd = up;
    }
}
```

3.3 Financial Loss Due to Unexpected TotalSurplus Status Update

SEVERITY:**HIGH****STATUS:****Fixed****PATH:**

PayMentNFT.sol

DESCRIPTION:

The `recaption()` function contains the statement `totalSurplus[round] = totalSurplus[round] = userReward;`, which does not reduce total surplus by the amount already paid out. As a result, `drawAbb()` later uses an understated denominator relationship and can distribute too much ABB.

```
function recaption(uint256 _tokenId,uint256 _round) external
nonReentrant{
    // ...
    if(_round==round){
        // ...
        tokenSurplus[_tokenId][round] = tokenSurplus[_tokenId][round]
-   userReward;
        userSurplus[msg.sender][round] =
userSurplus[msg.sender][round] - userReward;
        totalSurplus[round] = totalSurplus[round] = userReward;

    }
    // ...
}
```

This will cause `totalSurplus[round]` to be less than the actual value, and when `drawAbb()` retrieves expired `userSurplus[] []`, the result will be greater than the actual value.

```
function drawAbb(uint256 _round) external nonReentrant{
    require(totalSurplus[_round]>0,"no surplus");
    require(userSurplus[msg.sender][_round]>0,"you no surplus");
    require(_round!=round,"This round is not over yet");
    uint256 toUserAbb = abbAmount[_round] *
userSurplus[msg.sender][_round]/totalSurplus[_round];
    userSurplus[msg.sender][_round] = 0;
    abb.transfer(msg.sender, toUserAbb);
}
```

```
        emit DrawAbb(msg.sender, toUserAbb, _round, block.timestamp);  
    }
```

IMPACT:

In the `drawAbb()` function, the user received a reward exceeding the normal value due to a decrease in `totalSurplus`, resulting in a loss of contract funds.

RECOMMENDATIONS:

Subtract `userReward` from `totalSurplus[round]` instead of reassigning the current reward value.

```
-   totalSurplus[round] = totalSurplus[round] = userReward;  
+   totalSurplus[round] = totalSurplus[round] - userReward;
```

3.4 drawDynamic() Fails to Update totalSurplus[] Status

SEVERITY:**HIGH****STATUS:****Fixed****PATH:**

PayMentNFT.sol

DESCRIPTION:

`drawDynamic()` reduces `userSurplus[msg.sender][round]` but does not reduce `totalSurplus[round]`. This leaves stale aggregate accounting in place and suppresses later ABB settlement for all users.

```
function drawDynamic() external nonReentrant{
    if(userDynamic[msg.sender][round]>usdt.balanceOf(address(this))){
        boom();
    }else {
        uint256 totalReward = userDynamic[msg.sender][round];
        if(totalReward>userSurplus[msg.sender][round]){
            totalReward = userSurplus[msg.sender][round];
        }
        if(totalReward>0){
            totalDynamic[msg.sender] = totalDynamic[msg.sender] +
totalReward;
            uint256 toBuyAbb = totalReward / 10;
            uint256 getArt = totalReward / 20;
            uint256 toUser = totalReward - toBuyAbb - getArt;
            if(address(buyAbb) != address(0)){
                gotoBuyAbb(toBuyAbb, address(this));
            }
            usdt.transfer(toArt, getArt);
            usdt.transfer(msg.sender, toUser);
            userSurplus[msg.sender][round] =
userSurplus[msg.sender][round] - totalReward;
            userDynamic[msg.sender][round] = 0;
        }
        emit Draw(msg.sender, totalReward, round,block.timestamp,2);
    }
}
```

IMPACT:

When claiming dynamic rewards, `drawDynamic()` fails to synchronously decrement `totalSurplus`; this will ultimately result in a permanent miscalculation of the ABB settlement.

RECOMMENDATIONS:

Update `totalSurplus[round]` together with `userSurplus[msg.sender][round]` to keep aggregate and per-user accounting aligned.

```
function drawDynamic() external nonReentrant{
    // ...
    if(totalReward>0){
        // ...
        userSurplus[msg.sender][round] =
userSurplus[msg.sender][round] - totalReward;
+       totalSurplus[round] = totalSurplus[round] - totalReward;
        userDynamic[msg.sender][round] = 0;
    }
    emit Draw(msg.sender, totalReward, round,block.timestamp,2);
}
```


3.5 Missing userReward Update Leading to Transaction Revert

SEVERITY:**MEDIUM****STATUS:****Fixed****PATH:**

PayMentNFT.sol

DESCRIPTION:

In `recaption()`, if `userReward > usdt.balanceOf(address(this))`, the function calls `boom()` to advance the round, but it does not reset `userReward`. Because `userReward` remains greater than zero, the function continues into the transfer branch and reverts due to insufficient balance.

```
function recaption(uint256 _tokenId,uint256 _round) external
nonReentrant{
    // ...
    require(have == true,"You haven't staked this NFT yet");
    if(_round==round){
        uint256 rewardPerSec = 1e18 / uint256(86400);
        uint256 userReward = 0;
        userReward = (block.timestamp - lastUpdateTime[_tokenId]) *
rewardPerSec;
        if(userReward>tokenSurplus[_tokenId][round]){
            userReward = tokenSurplus[_tokenId][round];
        }
        if(userReward>userSurplus[msg.sender][round]){
            userReward = userSurplus[msg.sender][round];
        }
        tokenSurplus[_tokenId][round] = tokenSurplus[_tokenId][round]
- userReward;
        userSurplus[msg.sender][round] =
userSurplus[msg.sender][round] - userReward;
        totalSurplus[round] = totalSurplus[round] - userReward;
        if(userReward > usdt.balanceOf(address(this))){
            boom();
        }
        if(userReward>0){
            // ...
        }
    }
}
```

```
        usdt.transfer(toArt, getArt);
        usdt.transfer(msg.sender, toUser);
        usdt.transfer(msg.sender, userReward);
        emit Draw(msg.sender, userReward,
round,block.timestamp,1);
    }
    lastUpdateTime[_tokenId] = block.timestamp;
}
for (uint256 i = 0; i < arr.length; i++) {
    if(arr[i] == _tokenId){
        arr[i] = arr[arr.length-1];
        arr.pop();
        abbnft.transferFrom(address(this), msg.sender, _tokenId);
        emit Recaption(msg.sender,
_tokenId,round,block.timestamp);
        break;
    }
}

function boom() internal{
    boomToken = maxToken;
    round = round + 1;
    emit Boom(round-1,round,boomToken,block.timestamp);
}
```

IMPACT:

The transaction rollback prevented users from unstaking their NFTs, and the round could not be updated.

RECOMMENDATIONS:

When the contract balance is insufficient, set `userReward = 0` immediately after `boom()` and skip the reward payout branch.

```
function recaption(uint256 _tokenId,uint256 _round) external
nonReentrant{
    // ...
    require(have == true,"You haven't staked this NFT yet");
    if(_round==round){
```

```

        // ...
-         tokenSurplus[_tokenId][round] = tokenSurplus[_tokenId][round]
-         - userReward;
-         userSurplus[msg.sender][round] =
userSurplus[msg.sender][round] - userReward;
-         totalSurplus[round] = totalSurplus[round] = userReward;
        if(userReward > usdt.balanceOf(address(this))){
            boom();
+         userReward = 0;
        }
        if(userReward>0){
+         tokenSurplus[_tokenId][round] =
tokenSurplus[_tokenId][round] - userReward;
+         userSurplus[msg.sender][round] =
userSurplus[msg.sender][round] - userReward;
+         totalSurplus[round] = totalSurplus[round] = userReward;
        // ...
        usdt.transfer(toArt, getArt);
        usdt.transfer(msg.sender, toUser);
        usdt.transfer(msg.sender, userReward);
        emit Draw(msg.sender, userReward,
round,block.timestamp,1);
        }
        lastUpdateTime[_tokenId] = block.timestamp;
    }
    for (uint256 i = 0; i < arr.length; i++) {
        if(arr[i] == _tokenId){
            arr[i] = arr[arr.length-1];
            arr.pop();
            abbnft.transferFrom(address(this), msg.sender, _tokenId);
-         emit Recaption(msg.sender,
_tokenId,round,block.timestamp);
+         emit Recaption(msg.sender,
_tokenId,_round,block.timestamp);
            break;
        }
    }
}

```

3.6 addUserList() lacks deduplication check

SEVERITY:

LOW

STATUS:

Fixed

PATH:

PayMentNFT.sol

DESCRIPTION:

Administrators can add user addresses using `addUserList()`, but the function lacks duplicate checks. This can lead to the same address being inserted multiple times and distort subsequent reward distribution.

```
function addUserList(address _user) external onlyOwner{
    userList.push(_user);
}
```

IMPACT:

Duplicate addresses can affect the subsequent reward distribution in `toUserList()`, causing users to claim rewards repeatedly.

RECOMMENDATIONS:

Track whether an address has already been added and reject duplicate inserts. Clear that state again when the address is removed.

Set a mapping variable to record state changes and check for duplicates.

```
+ mapping(address => bool) public isAdded;
  address[] public userList;

  function addUserList(address _user) external onlyOwner{
+   require(!isAdded[_user], "User already in list");
    userList.push(_user);
+   isAdded[_user] = true;
  }
```

```
function removeUserList(address _user) external onlyOwner{  
+   require(isAdded[_user], "User not in list");  
    bool a = false;  
    for(uint256 i;i<userList.length;i++){  
        if (userList[i]==_user){  
            userList[i] = userList[userList.length-1];  
            userList.pop();  
+           isAdded[_user] = false;  
            a = true;  
            break;  
        }  
    }  
    require(a==true,"no user");  
}
```

3.7 Unchecked ERC20 Transfer Return Values

SEVERITY: LOW**STATUS:** Fixed**PATH:**

PayMentNFT.sol

DESCRIPTION:

The contract performs multiple `ERC20.transfer()` calls without checking their return values. If the token returns `false` instead of reverting, the contract may continue execution even though the transfer failed, causing accounting inconsistencies and failed payouts.

```
usdt.transfer(toArt, getArt);  
usdt.transfer(msg.sender, toUser);  
usdt.transfer(msg.sender, userReward);
```

IMPACT:

Failed token transfers may not revert, causing the contract to continue with updated state while no funds are actually transferred. This can lead to accounting inconsistencies, incorrect reward distribution, and user fund loss.

RECOMMENDATIONS:

Adopt OpenZeppelin `SafeERC20` and replace raw token transfer calls with safe wrappers so failed transfers always revert.

```
-import "../interface/IERC20.sol";  
+import "@openzeppelin/contracts/token/ERC20/IERC20.sol";  
+import "@openzeppelin/contracts/token/ERC20/utils/SafeERC20.sol";  
  
contract payNft is WhiteList, ReentrancyGuard {  
+    using SafeERC20 for IERC20;  
}
```

4. CONCLUSION

In this audit, we thoroughly analyzed **PayMentNFT** smart contract implementation. The problems found are described and explained in detail in Section 3. The problems found in the audit have been communicated to the project leader. We therefore consider the audit result to be **PASSED**.

To improve this report, we greatly appreciate any constructive feedbacks or suggestions, on our methodology, audit findings, or potential gaps in scope/coverage.

5. APPENDIX

5.1 Basic Coding Assessment

5.1.1 Apply Verification Control

Description	The security of apply verification
Result	Not found
Severity	CRITICAL

5.1.2 Authorization Access Control

Description	Permission checks for external integral functions
Result	Not found
Severity	CRITICAL

5.1.3 Forged Transfer Vulnerability

Description	Assess whether there is a forged transfer notification vulnerability in the contract
Result	Not found
Severity	CRITICAL

5.1.4 Transaction Rollback Attack

Description	Assess whether there is transaction rollback attack vulnerability in the contract
Result	Not found
Severity	CRITICAL

5.1.5 Transaction Block Stuffing Attack

Description	Assess whether there is transaction blocking attack vulnerability
Result	Not found
Severity	CRITICAL

5.1.6 Soft Fail Attack Assessment

Description	Assess whether there is soft fail attack vulnerability
Result	Not found
Severity	CRITICAL

5.1.7 Hard Fail Attack Assessment

Description	Examine for hard fail attack vulnerability
Result	Not found
Severity	CRITICAL

5.1.8 Abnormal Memo Assessment

Description	Assess whether there is abnormal memo vulnerability in the contract
Result	Not found
Severity	CRITICAL

5.1.9 Abnormal Resource Consumption

Description	Examine whether abnormal resource consumption in contract processing
Result	Not found
Severity	CRITICAL

5.1.10 Random Number Security

Description	Examine whether the code uses insecure random number
Result	Not found
Severity	CRITICAL

5.2 Advanced Code Scrutiny

5.2.1 Cryptography Security

Description	Examine for weakness in cryptograph implementation
Result	Not found
Severity	HIGH

5.2.2 Account Permission Control

Description	Examine permission control issue in the contract
Result	Not found
Severity	MEDIUM

5.2.3 Malicious Code Behavior

Description	Examine whether sensitive behavior present in the code
Result	Not found
Severity	MEDIUM

5.2.4 Sensitive Information Disclosure

Description	Examine whether sensitive information disclosure issue present in the code
Result	Not found
Severity	MEDIUM

5.2.5 System API

Description	Examine whether system API application issue present in the code
Result	Not found
Severity	LOW

6. DISCLAIMER

This report is subject to the terms and conditions (including without limitation, description of services, confidentiality, disclaimer and limitation of liability) set forth in the Services Agreement, or the scope of services, and terms and conditions provided to the Company in connection with the Agreement. This report provided in connection with the Services set forth in the Agreement shall be used by the Company only to the extent permitted under the terms and conditions set forth in the Agreement. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes without ExVul's prior written consent.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts ExVul to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. ExVul's position is that each company and individual are responsible for their own due diligence and continuous security. ExVul's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

7. REFERENCES

- [1] MITRE. CWE-191: Integer Underflow (Wrap or Wraparound). <https://cwe.mitre.org/data/definitions/191.html>.
 - [2] MITRE. CWE-197: Numeric Truncation Error. <https://cwe.mitre.org/data/definitions/197.html>.
 - [3] MITRE. CWE-400: Uncontrolled Resource Consumption. <https://cwe.mitre.org/data/definitions/400.html>.
 - [4] MITRE. CWE-440: Expected Behavior Violation. <https://cwe.mitre.org/data/definitions/440.html>.
 - [5] MITRE. CWE-684: Protection Mechanism Failure. <https://cwe.mitre.org/data/definitions/693.html>.
 - [6] MITRE. CWE CATEGORY: 7PK - Security Features. <https://cwe.mitre.org/data/definitions/254.html>.
 - [7] MITRE. CWE CATEGORY: Behavioral Problems. <https://cwe.mitre.org/data/definitions/438.html>.
 - [8] MITRE. CWE CATEGORY: Numeric Errors. <https://cwe.mitre.org/data/definitions/189.html>.
 - [9] MITRE. CWE CATEGORY: Resource Management Errors. <https://cwe.mitre.org/data/definitions/399.html>.
 - [10] OWASP. Risk Rating Methodology. https://www.owasp.org/index.php/OWASP_Risk_Rating_Methodology
-

8. About Exvul Security

Premier Security for the Web3 Ecosystem

ExVul is a premier Web3 security firm committed to forging a secure and trustworthy decentralized ecosystem. Our elite team consists of security veterans from world-leading technology and blockchain security firms, including Huawei, YBB Captical, Qihoo 360, Amber, ByteDance, MoveBit, and PeckShield. Team member Nolan is ranked as a top-40 whitehat on Immunefi and is the platform's sole All-Star in the APAC region.

Our expertise covers the full spectrum of Web3 security. We conduct **meticulous smart contract audits**, having fortified thousands of projects on chains like Evm, Solana, Aptos, Sui etc. Our **Blockchain Protocol Audits** secure the core infrastructure of L1/L2 by uncovering deep-seated vulnerabilities. We also offer **comprehensive wallet audits** to protect user assets and provide **proactive web3 pentest**, enabling partners to neutralize threats before they strike.

Trusted by industry leaders, ExVul is the security partner for **OKX, Bitget, Cobo, Infini, Stacks, Aptos, Sui, CoreDAO, Sei** etc.

Contact

 **Website**
www.exvul.com

 **Twitter**
[@EXVULSEC](https://twitter.com/EXVULSEC)

 **Email**
contact@exvul.com

 **Github**
github.com/EXVUL-Sec